

{ }

< / >

Java
Today
A Better
Tomorrow



Code
Ideas
Progress

Learn
Practice
Build



Programming Language



Language

- Any medium to communicate is called a language.



Programming Language

- Any language that is used to communicate with a computer or a machine is called Programming Language. Programming language is the form of set of instructions.



Types of Programming Language

1

Low Level (Binary Language [0,1]).

- A computer can directly understand assembly language.

2

Assembly Language (Mod, Bit and other)

- It is in the form of mnemonics (easily understood words).
- It is directly converted into machine language by an assembler.



James Gosling and Patrick Naughton

Employees of Sun Microsystems, with support from **Bill Joy** — co-founders of Sun Microsystems.



Low Level Language

- Low level language is the first programming language which was used to instruct computer.
- It is in binary form [0,1], so it is called **Binary Language**.
- A computer can directly understand low level language, but it is difficult for a human to understand.

e.g.:

0	→	off
1	→	on



Assembly Language vs. Machine/Low Level Language

Assembly Language

- A computer can directly understand assembly language.
- It is in the form of mnemonics (easily understood words).
- It is directly converted into machine language by an assembler.

Machine/Low Level Language

- A computer can directly understand machine language.
- It is in the form of 0s and 1s (binary).
- It is not easy for a human to understand.
- It is directly executed by the computer.



Key Point:

A computer cannot understand assembly language directly. So, assembly language is used as a translator which takes assembly language and converts it into low level language (machine language).



High Level Language

- A language that a human can understand directly is called high level language.
- It is a high level language because it is easy to understand.
- It makes programming very easy to understand.

e.g.: 1972 → C language

A computer cannot understand high level language code directly, so compiler and interpreter are used. In C, a translator which takes high level code and converts into low level code is used.



Every high level language will have own compiler.

* Java :

- Java is a high level, object oriented language [platform independent] computer programming language, which features object oriented concepts [Encapsulation, Inheritance, Polymorphism, Abstraction].

* Java was developed by James Gosling in 1991 under Sun Microsystems and it was released in 1995.

1995 ⇒ Java beta version.

1996 ⇒ Java 1.0.0 (JDK 1.0.0).

1997 ⇒ Java 2.0.0 (JDK 2.0.0).

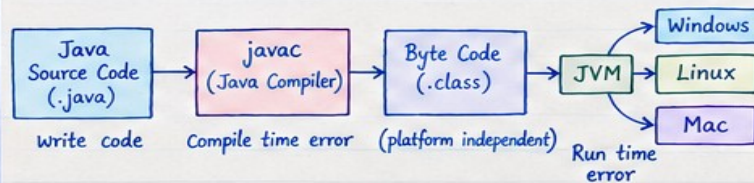
⋮

March 2022 ⇒ Java 18.0.0 (JDK 18.0.0).

→ 20

* In 2010 ORACLE bought Sun Microsystems.

* Java is a platform independent programming language.

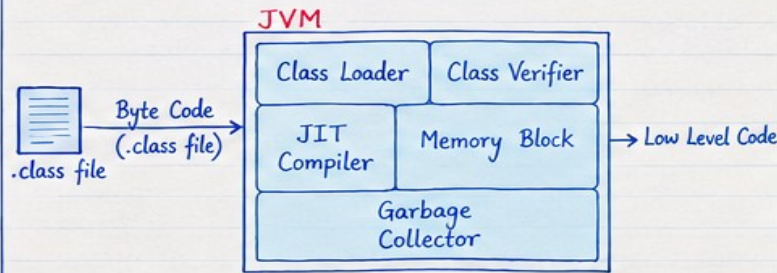


* When an application is developed by using Java language then the .java file is saved as first step and it is called Java Compilation, which takes .java file and if it is correct then next level code will be generated by compiler which is called Byte Code or .class file. Byte Code is also called Magical or Intermediate Code, which a computer or JVM will understand directly.

* JVM : JVM stands for Java Virtual Machine. It works as a translator which takes byte code and converts into the local code, which a computer can understand. In this manner Java becomes a platform independent. So it is called "WORA" [Write Once RUN Anywhere] language.

* JRE, JDK, JIT Compiler :

1) Java Virtual Machine :



⇒ JVM stands for Java Virtual Machine. JVM is called Virtual because JVM doesn't exist alone. JVM needs a body for implementation.

2) The Components of JVM are :

(a) Class Loader : Loads the class file inside JVM.

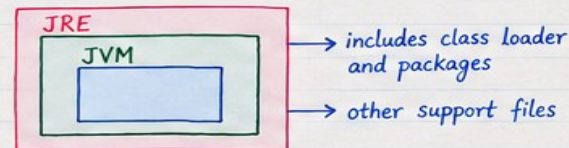
(b) Class Verifier : Will reads loaded class file line by line and checks that generated byte codes are correct. So that overall efficiency can be improved.

(c) JIT Compiler : Used to convert byte code to native code to improve the performance.

(d) Memory Block : Takes memory area for all memory related tasks.

(e) Garbage Collector : Deletes unwanted memory so that execution can be maintain.

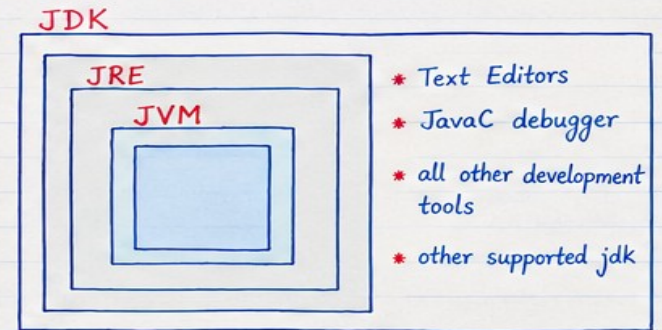
* JRE :



JRE stands for Java Runtime Environment. JRE provides implementation to JVM. So JVM is physically present inside JRE. JRE containing JVM along with class files and packages and library files. By using JRE we can only run the applications but we can't do development parts.

* JDK (Java Development Kit) :

1) Java Virtual Machine :



JDK stands for Java Development Kit. It combines JRE along with development tools which are needed by a programmer to develop an application. By using JDK we can run the application as well as we can do development part.

JDK is platform dependent So Different platform have different JDK.

* JIT (Just In Time) Compiler :

JIT is a part of JVM. It is used inside JVM to improve its efficiency.

* JRE, JDK, JIT Compiler :

① JRE - provides runtime environment to JVM.

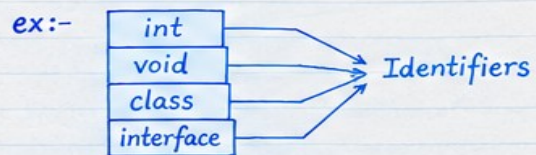
② JDK - provides JRE along with development tools and other support files.

③ JIT - used inside JVM to improve efficiency.

* Identifiers :

In Java, identifiers are the names of classes, variables, methods, constructors, packages, interfaces or any member in the program. Identifiers are used to give a name to a program element.

→ Keywords are special types of identifiers but they are not used as identifiers. Identifiers are used only for identification purpose.



Note: Every identifier must be a valid name (not a keyword).

* Rules for Identifiers :

- ① For identifiers, we can use alphabets (a-z or A-Z), numbers (0-9) and special characters (_ and \$) but not start with a number.
- ② Identifiers must not start with a number.
- ③ No space allowed for identifiers.
- ④ Keywords cannot be used as an identifier.

eg:-

1) FirstProgram123	✓
2) FirstProgram@123	✗
3) FirstProgram\$123	✓
4) 123FirstProgram	✗
5) _123FirstProgram	✓
6) \$FirstProgram	✓
7) First Program	✗
8) public	✗
9) Public	✗
10) Public1	✓

→ java is a case sensitive language so uppercase and lowercase are different.

* Conventions for Identifiers :

Conventions are set of guidelines for identifier for good programming practice.

* Conventions for a Class Name :

- Class name should be written in 'UpperCamelCase'.
- It should start with a capital letter.
- Example: StudentRecord, Employee, FirstProgram

* Conventions for a Method Name :

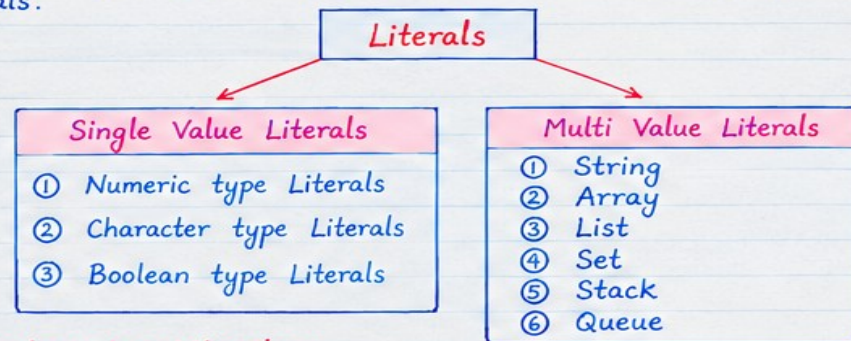
- Method name should be written in 'lowerCamelCase'.
- It should start with a small letter.
- Example: displayInfo(), calculateSum(), getData()

* Conventions for a Variable Name :

- Variable name should be written in 'lowercase'.
- It should start with a small letter.
- Example: age, name, price, total

* Literals :

The value of different type of data is stored in a program is called Literals.



① Single Value Literals :

- Numeric type Literals :
Represents numeric value (integer, floating point, etc).
Example: 10, 20, 3.14, 0, -5
- Character type Literals :
Represents a single character and written in single quotes.
Example: 'a', 'z', '1', '\$'
- Boolean type Literals :
Represents boolean value (true or false).
Example: true, false

* 1. Data :

Data is any piece of information. In Java, data are the names of constants, variables, and other values used in a program.

* 2. Data Types :

In Java, data types are the names of classes, variables, methods, constructors, packages, interfaces or any member in the program. Identifiers are used to give a name to a program element.

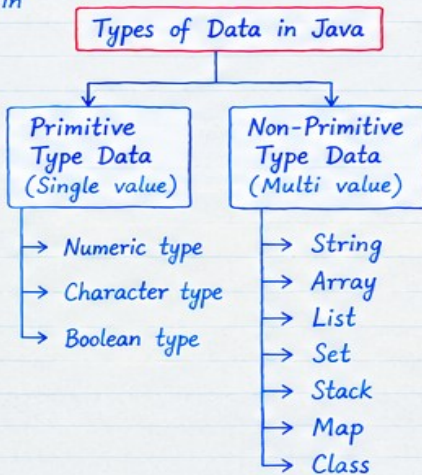
→ Keywords are special types of identifiers but they are not used as identifiers. Identifiers are used only for identification purpose.

* 3. Types of Data :

In Java, data are divided into two types :

(a) Primitive type data (Single value data)

(b) Non-primitive type data (Multi-value data)



Design by Sarafraj Navaz

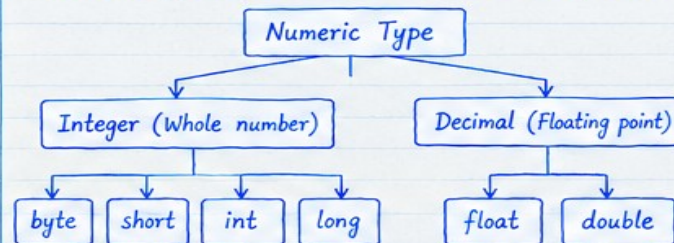
* 4. Primitive Type Data :

Primitive types are the basic building blocks of data. They are single value data types and are divided into:

1) Numeric type 2) Character type 3) Boolean type

(a) Numeric Type :

Numeric type is further divided into two categories :



Design by Sarafraj Navaz

(b) Character Type :

Character type is used to store single character. It is represented by 16 bit.

Example : `char ch = 'A';`

(c) Boolean Type :

Boolean type is used to store true or false value.

Example : `boolean flag = true;`

* 5. Non-Primitive Type Data :

Non-primitive types store multiple values or complex data structures.

Example : String, Array, List, Set, Stack, Map, Class.

* 6. Data Type Size and Range :

Data Type	Size (bits)	Size (bytes)	Range
boolean	1	1 bit	0 to 1
byte	8	1 byte	-128 to 127
short	16	2 bytes	-32,768 to 32,767
int	32	4 bytes	-2,147,483,648 to 2,147,483,647
long	64	8 bytes	-9,223,372,036,854,775,808 to 9,223,372,036,854,775,807
float	32	4 bytes	-3.402E38 to 3.402E38 (6-7 digits)
double	64	8 bytes	-1.797E308 to 1.797E308 (15-16 digits)
char	16	2 bytes	0 to 65,535 (Unicode)

* 7. Character Representation :

ASCII :

- Used for English characters.
- Range : 0 - 127.
- Example : 'A' = 65

Unicode :

- Used for all languages and symbols.
- Range : 0 - 65,535 (16 bit).
- Example : 'A' = 65, 'अ' (Hindi) = 2309

Design by Sarafraj Navaz

* 8. Variables :

A variable is a container that holds a value. The value can be changed during program execution.

* 9. Declaration, Initialization and Declaration + Initialization :

- 1) Declaration : `int age;`
- 2) Initialization : `age = 25;`
- 3) Declaration + Initialization : `int age = 25;`

* 10. Types of Variables :

